

TeRun - Territory Run

Technische Dokumentation: LoginScreen.kt

Mobile Computing SoSe 2026 | Prof. Dr. Matthias Boehmer | TH Koeln

1. Überblick

Die Datei LoginScreen.kt implementiert den ersten Screen der TeRun-App. Es handelt sich um einen Splash-Screen mit Logo, App-Name und zwei Buttons. Der Screen ist vollständig in Jetpack Compose geschrieben und folgt dem deklarativen UI-Paradigma das in der Vorlesung vermittelt wurde.

Der LoginScreen ist der startDestination der App - der erste Screen den ein Nutzer sieht wenn er TeRun oeffnet.

2. Quellen

2.1 Vorlesungsunterlagen (Primaerquellen)

- moco202611declarativeui.pdf - Deklaratives UI-Paradigma, @Composable Annotation
- moco202612creatingcomposables.pdf - Column, Button, Text, @Preview
- moco202613composablesmodifier.pdf - Modifier, fillMaxSize, background, padding, clip, size
- moco202617pixeldensities.pdf - Verwendung von dp und sp statt px

2.2 Externe Quellen

- Button: <https://developer.android.com/develop/ui/compose/components/button>
- Canvas und Path:
<https://developer.android.com/develop/ui/compose/graphics/draw/overview>

3. Aufbau des Screens

Composable	Aufgabe	Quelle
Box (aussen)	Fuellt den ganzen Bildschirm mit dunklem Hintergrund	moco202613
Column	Stapelt alle Elemente untereinander, zentriert	moco202612
Box (Logo)	Blaues abgerundetes Quadrat als Logo-Container	moco202613
Canvas	Zeichnet das stilisierte T mit schraeger Luecke	developer.android.com

<code>Text (TeRun)</code>	Zeigt den App-Namen in weisser fetter Schrift	moco202612
<code>Text (Territory Run)</code>	Untertitel in kleiner halbtransparenter Schrift	moco202612
<code>Button</code>	Blauer ausgefuellter Primaer-Button	moco202612
<code>OutlinedButton</code>	Transparenter Button mit weissem Rand	developer.android.com
<code>Spacer</code>	Unsichtbarer Abstandshalter zwischen Elementen	moco202613

4. Code-Erklaerung Zeile fuer Zeile

4.1 package com.example.terun

Definiert das Paket dieser Datei. Jede Kotlin-Datei in Android gehoert zu einem Paket. Das Paket entspricht dem Ordnerpfad im Projekt und hilft Android dabei die Datei zu finden und zuzuordnen.

4.2 import-Anweisungen

```
import androidx.compose.foundation.Canvas
```

Importiert den Canvas-Composable. Canvas ist eine Zeichenflaeche auf der man mit Path und drawPath eigene Grafiken erstellen kann. Wird fuer das stilisierte T-Logo verwendet. Quelle: developer.android.com/draw

```
import androidx.compose.foundation.background
```

Importiert den background-Modifier. Damit kann man jedem Composable eine Hintergrundfarbe geben. Quelle: [moco202613composablesmodifier.pdf](#)

```
import androidx.compose.foundation.layout.*
```

Importiert alle Layout-Composables auf einmal - Box, Column, Row, Spacer und alle Modifier wie fillMaxSize, padding, height, size. Das Sternchen (*) bedeutet: importiere alles aus diesem Paket.

```
import androidx.compose.foundation.shape.RoundedCornerShape
```

Importiert RoundedCornerShape. Diese Klasse beschreibt eine Form mit abgerundeten Ecken. Wird fuer das Logo-Quadrat und die Buttons verwendet.

```
import androidx.compose.material3.Button
```

Importiert den Material Design 3 Button-Composable. Material Design ist Googles offizielle Design-Sprache fuer Android. Quelle: [moco202612creatingcomposables.pdf](#)

```
import androidx.compose.material3.ButtonDefaults
```

Importiert ButtonDefaults - eine Klasse mit Standard-Einstellungen fuer Buttons. Damit kann man Farben des Buttons ueberschreiben, zum Beispiel containerColor fuer die Hintergrundfarbe.

```
|import androidx.compose.material3.OutlinedButton
```

Importiert den OutlinedButton - ein Button ohne Hintergrundfuellung, nur mit einem Rand. Wird fuer den zweiten Button verwendet. Quelle: developer.android.com/button

```
|import androidx.compose.material3.Text
```

Importiert den Text-Composable. Text zeigt Zeichenketten auf dem Bildschirm an. Alle sichtbaren Texte in der App werden mit Text dargestellt. Quelle: moco202612creatingcomposables.pdf

```
|import androidx.compose.runtime.Composable
```

Importiert die @Composable-Annotation. Ohne diese Annotation kann eine Funktion nicht als UI-Funktion verwendet werden. Jede Funktion die Composables aufruft muss mit @Composable annotiert sein. Quelle: moco202611declarativeui.pdf

```
|import androidx.compose.ui.Alignment
```

Importiert Alignment - eine Klasse fuer Ausrichtungsoptionen. Alignment.Center zentriert den Inhalt einer Box. Alignment.CenterHorizontally zentriert ihn waagerecht in einer Column.

```
|import androidx.compose.ui.Modifier
```

Importiert Modifier - das wichtigste Konzept fuer Layout und Styling in Jetpack Compose. Modifier ist eine Kette von Anweisungen die Aussehen und Verhalten eines Composables steuern. Quelle: moco202613composablesmodifier.pdf

```
|import androidx.compose.ui.draw.clip
```

Importiert den clip-Modifier. clip schneidet ein Composable in eine bestimmte Form - in diesem Fall in ein abgerundetes Quadrat fuer das Logo. Quelle: moco202613composablesmodifier.pdf

```
|import androidx.compose.ui.graphics.Color
```

Importiert die Color-Klasse. Farben werden als Color-Objekte definiert. Der Hex-Wert 0xFF steht fuer volle Deckkraft, gefolgt von RRGGBB. Beispiel: 0xFF1A6FF5 ist das TeRun-Blau.

```
|import androidx.compose.ui.graphics.Path
```

Importiert Path - eine Klasse die einen geometrischen Pfad beschreibt. Mit moveTo, lineTo und cubicTo werden Punkte definiert die das T-Logo zeichnen. Quelle: developer.android.com/draw

```
|import androidx.compose.ui.text.font.FontWeight
```

Importiert FontWeight fuer Schriftgewichte. FontWeight.Bold macht Text fett. FontWeight.SemiBold ist leichter. FontWeight.Medium ist die mittlere Staerke.

```
|import androidx.compose.ui.tooling.preview.Preview
```

Importiert die `@Preview`-Annotation. Preview zeigt den Composable direkt in Android Studio ohne App-Start. Quelle: [moco202612creatingcomposables.pdf](#)

```
|import androidx.compose.ui.unit.dp
```

Importiert `dp` - density-independent pixels. Immer `dp` fuer Abstaende und Groessen verwenden, niemals `px`. Auf einem hochaufloesenden Display wird `1dp` automatisch groesser dargestellt. Quelle: [moco202617pixeldensities.pdf](#)

```
|import androidx.compose.ui.unit.sp
```

Importiert `sp` - scalable pixels. `sp` ist wie `dp` aber fuer Schriftgroessen. `sp` beruecksichtigt zusaetzlich die vom Nutzer eingestellte Systemschriftgroesse. Quelle: [moco202617pixeldensities.pdf](#)

4.3 Farbkonstanten

```
|val DarkBackground = Color(0xFF0D1B2A)
```

Definiert die Hintergrundfarbe als Konstante. `val` bedeutet in Kotlin dass der Wert nicht veraendert werden kann. `0xFF` ist der Alpha-Kanal fuer volle Deckkraft, `0D1B2A` ist der Hex-Farbcode fuer dunkles Blau-Schwarz.

```
|val TeRunBlue = Color(0xFF1A6FF5)
```

Definiert das TeRun-Blau als Konstante. Dieser Wert wird an mehreren Stellen verwendet - Logo-Hintergrund und Primaer-Button. Durch die Konstante muss man die Farbe nur an einer Stelle aendern wenn man sie anpassen moechte.

4.4 LoginScreen Funktion

```
|@Composable
```

Die `@Composable`-Annotation teilt dem Kotlin-Compiler mit dass diese Funktion UI beschreibt. Composable-Funktionen geben nichts zurueck - sie beschreiben nur wie der Bildschirm aussehen soll. Quelle: [moco202611declarativeui.pdf](#)

```
|fun LoginScreen(onStartClicked: () -> Unit, onLoginClicked: () -> Unit)
```

Definiert die LoginScreen-Funktion mit zwei Parametern. Beide Parameter sind Funktionen die keinen Rueckgabewert haben `() -> Unit`. `onStartClicked` wird aufgerufen wenn der Nutzer Jetzt starten drueckt. `onLoginClicked` wenn er Bereits registriert drueckt. Der Screen delegiert die Navigation nach aussen und bleibt dadurch wiederverwendbar.

4.5 Box - aeusserer Container

```
|Box(modifier = Modifier.fillMaxSize().background(DarkBackground),  
|contentAlignment = Alignment.Center)
```

Box ist ein Container in dem Kindelemente uebereinandergelegt werden koennen.
fillMaxSize() macht die Box so gross wie der gesamte Bildschirm.
background(DarkBackground) faerbt den Hintergrund dunkel. contentAlignment = Alignment.Center zentriert alles darin sowohl waagerecht als auch senkrecht. Quelle: moco202613composablesmodifier.pdf

4.6 Column - vertikales Layout

```
Column(horizontalAlignment = Alignment.CenterHorizontally,  
verticalArrangement = Arrangement.Center, modifier =  
Modifier.fillMaxWidth().padding(horizontal = 32.dp).padding(bottom = 80.dp))
```

Column stapelt alle Kindelemente untereinander. horizontalAlignment = Alignment.CenterHorizontally zentriert alle Elemente waagerecht. verticalArrangement = Arrangement.Center schiebt alle Elemente in die vertikale Mitte. padding(horizontal = 32.dp) gibt links und rechts 32dp Abstand. padding(bottom = 80.dp) schiebt den Inhalt etwas nach oben. Quelle: moco202612creatingcomposables.pdf, moco202613composablesmodifier.pdf

4.7 Logo-Box

```
Box(modifier =  
Modifier.size(88.dp).clip(RoundedCornerShape(20.dp)).background(TeRunBlue),  
contentAlignment = Alignment.Center)
```

Diese innere Box ist der blaue Logo-Container. size(88.dp) setzt Breite und Hoehe auf 88dp. clip(RoundedCornerShape(20.dp)) schneidet die Ecken mit einem Radius von 20dp ab und ergibt das abgerundete Quadrat. background(TeRunBlue) faerbt es blau. Die Reihenfolge der Modifier ist wichtig: clip muss vor background stehen damit der Hintergrund auch abgerundet wird. Quelle: moco202613composablesmodifier.pdf

4.8 Canvas - das T-Logo

```
Canvas(modifier = Modifier.size(56.dp))
```

Canvas ist eine Zeichenflaeche mit 56dp Groesse. Innerhalb von Canvas erstellt man eigene Grafiken mit Zeichenbefehlen. size.width und size.height geben die Breite und Hoehe der Zeichenflaeche zurueck. Quelle: developer.android.com/develop/ui/compose/graphics/draw/overview

```
val querbalkenLinks = Path().apply { moveTo(...).lineTo(...) close() }
```

Path() erstellt einen leeren geometrischen Pfad. moveTo(x, y) setzt den Startpunkt ohne zu zeichnen..lineTo(x, y) zeichnet eine gerade Linie zum angegebenen Punkt. close() schliesst den Pfad indem eine Linie zurueck zum Startpunkt gezogen wird. Alle Koordinaten sind Prozentwerte der Canvas-Groesse - w mal 0.18f bedeutet 18 Prozent der Breite.

Das T besteht aus drei Pfaden: querbalkenLinks ist der linke Teil des Querbalkens bis zur schraegen Luecke bei 18 bis 22 Prozent. querbalkenRechts ist der rechte Teil nach der Luecke. senkrecht ist der schraege Strich der nach unten links verlauft und dem Logo seinen sportlichen Charakter gibt.

```
drawPath(querbalkenLinks, color = Color.White)
```

drawPath zeichnet den definierten Pfad auf die Canvas. color = Color.White füllt die Form mit weisser Farbe. Ohne style-Parameter wird die Form ausgefüllt dargestellt - nicht als Linie.

4.9 Text - App-Name

```
Text(text = "TeRun", fontSize = 42.sp, fontWeight = FontWeight.Bold, color = Color.White)
```

Text zeigt den App-Namen an. fontSize = 42.sp setzt die Schriftgrösse auf 42 scalable pixels. fontWeight = FontWeight.Bold macht den Text fett. color = Color.White setzt die Schriftfarbe auf Weiss. Quelle: moco202612creatingcomposables.pdf

4.10 Text - Untertitel

```
Text(text = "Territory Run", fontSize = 17.sp, color = Color.White.copy(alpha = 0.45f))
```

Der Untertitel ist kleiner und halbtransparent. Color.White.copy(alpha = 0.45f) nimmt Weiss und setzt die Transparenz auf 45 Prozent. Das ergibt ein gedämpftes Grau auf dunklem Hintergrund ohne eine neue Farbe definieren zu müssen.

4.11 Spacer

```
Spacer(modifier = Modifier.height(20.dp))
```

Spacer ist ein unsichtbares Composable das nur Platz einnimmt. height(20.dp) setzt die Hoehe auf 20dp. Spacer wird zwischen Elementen verwendet um Abstaende zu erzeugen ohne die Elemente selbst zu veraendern. Quelle: moco202613composablesmodifier.pdf

4.12 Button - Primaer-Button

```
Button(onClick = onStartClicked, modifier = Modifier.fillMaxWidth().height(60.dp), shape = RoundedCornerShape(26.dp), colors = ButtonDefaults.buttonColors(containerColor = TeRunBlue))
```

Button ist der ausgefüllte blaue Primaer-Button. onClick = onStartClicked verbindet den Button mit der uebergebenen Funktion. fillMaxWidth() macht den Button so breit wie der verfuegbare Platz. height(60.dp) setzt die Hoehe. shape = RoundedCornerShape(26.dp) ergibt bei 60dp Hoehe eine Kapsel-Form. colors = ButtonDefaults.buttonColors(containerColor = TeRunBlue) setzt die Hintergrundfarbe. Quelle: moco202612creatingcomposables.pdf, developer.android.com/button

4.13 OutlinedButton - Ghost-Button

```
OutlinedButton(onClick = onLoginClicked, modifier = ..., border = BorderStroke(1.5.dp, Color.White.copy(alpha = 0.4f)))
```

OutlinedButton ist ein Button ohne Hintergrundfuellung mit nur einem sichtbaren Rand. BorderStroke(1.5.dp, ...) definiert den Rand mit 1.5dp Breite und halbtransparentem Weiss. colors = ButtonDefaults.outlinedButtonColors(contentColor = Color.White) setzt die Textfarbe auf Weiss. Quelle: developer.android.com/button

4.14 Preview

```
|@Preview(showBackground = true)
```

@Preview sagt Android Studio: zeige diesen Composable in der Design-Ansicht ohne App-Start auf einem Gerät. showBackground = true zeigt einen Hintergrund damit man die Preview besser sieht. Die Preview-Funktion ruft LoginScreen mit leeren Funktionen auf da Navigation in der Preview nicht benötigt wird. Quelle: moco202612creatingcomposables.pdf

5. Wichtige Konzepte fuer die Pruefung

5.1 Deklaratives UI

Jetpack Compose ist deklarativ. Man beschreibt wie der Bildschirm aussehen soll - nicht welche Schritte nötig sind um ihn zu bauen. Im imperativen Ansatz würde man sagen: erstelle einen Button, setze seine Farbe auf Blau, füge ihn zur View hinzu. In Compose sagt man: es gibt einen blauen Button. Das System kümmert sich um den Rest. Quelle: moco202611declarativeui.pdf

5.2 Modifier-Reihenfolge

Die Reihenfolge der Modifier ist in Jetpack Compose entscheidend. Modifier werden von oben nach unten angewendet. Bei clip und background ist die Reihenfolge wichtig: clip muss zuerst kommen damit der Hintergrund innerhalb der geclippten Form gezeichnet wird. Bei clickable und padding ist die Reihenfolge sichtbar unterschiedlich - je nachdem ob das padding vor oder nach clickable kommt verändert sich die klickbare Fläche. Quelle: moco202613composablesmodifier.pdf

5.3 dp vs sp vs px

px ist geräteabhängig - auf einem hochauflösenden Display wäre 1px sehr klein. dp ist geräteunabhängig - 1dp sieht auf allen Geräten gleich gross aus weil Android es automatisch skaliert. sp ist wie dp aber berücksichtigt zusätzlich die Schriftgrösseneinstellung des Nutzers. Regel: immer dp für Grössen und Abstände, immer sp für Schriftgrössen. Quelle: moco202617pixeldensities.pdf

5.4 Funktionen als Parameter

LoginScreen empfängt zwei Parameter die selbst Funktionen sind: onStartClicked und onLoginClicked. Das ist ein wichtiges Kotlin-Konzept - Funktionen sind in Kotlin sogenannte first-class citizens, sie können als Parameter übergeben werden. Der Screen weiss dadurch nicht wohin er navigiert - das entscheidet die übergeordnete Funktion die LoginScreen aufruft. So bleibt der LoginScreen wiederverwendbar und von der Navigation entkoppelt.

6. Zusammenfassung

Der LoginScreen ist der erste Screen von TeRun und wurde vollstaendig in Jetpack Compose implementiert. Er besteht aus einer Box als Hintergrund-Container, einer Column fuer das vertikale Layout, einer Logo-Box mit Canvas-gezeichnetem T mit schraeger Luecke, zwei Text-Composables fuer Name und Untertitel sowie einem Button und einem OutlinedButton fuer die Benutzerinteraktion. Alle verwendeten Konzepte - Composables, Modifier, dp und sp, @Preview - stammen direkt aus den Vorlesungsunterlagen von Prof. Dr. Matthias Boehmer (TH Koeln, Mobile Computing SoSe 2026).